

VECTOR:

The load operation has a time complexity of $O(n)$, searching is $O(n)$, sorting is $O(n \log n)$, and printing sorted data is $O(n)$. A vector is a simple and easy-to-implement data structure that is suitable for beginners. However, it has slower search performance compared to the other data structures and requires an additional sorting step before printing data in order.

HASH TABLE:

The hash table has an insertion time of $O(1)$ on average and a search time of $O(1)$ on average. In the worst case, due to collisions, both operations can degrade to $O(n)$. Printing in sorted order requires $O(n \log n)$ time because the data must first be extracted and sorted. Overall, the hash table provides efficient lookup performance. However, it does not maintain a sorted order, which requires additional processing when ordered output is needed.

BINARY SEARCH TREE:

The binary search tree has an insertion time of $O(\log n)$ on average and $O(n)$ in the worst case. Searching also has $O(\log n)$ average-case performance and $O(n)$ worst-case performance. In-order traversal allows printing in sorted order in $O(n)$ time. While it maintains sorted structure naturally, performance can degrade significantly if the tree becomes unbalanced. Additionally, for lookup-heavy workloads, it is generally less efficient than a hash table.

CONCLUSION:

Although vectors and binary search trees both provide useful features, the hash table is the most appropriate data structure for this project. The program relies heavily on frequent lookup

operations for validating prerequisites, and the hash table provides $O(1)$ average-case search time, making it the most efficient choice for this requirement.

While a binary search tree maintains sorted order and supports efficient traversal, its performance can degrade to $O(n)$ in the worst case and is less efficient than a hash table for repeated lookups. A vector is the simplest structure, but its $O(n)$ search time and need for sorting make it unsuitable for a lookup-intensive application.